

SCALABLE MICROSERVICES ARCHITECTURE WITH ECS AND FARGATE

Services: ECS, Fargate, ECR, RDS, CloudWatch

Description: Containerize a microservices-based application using Docker. Deploy using ECS Fargate and manage logs with CloudWatch. Use RDS as the backend database.

Skills: Containers, microservices, monitoring

Definition

This project demonstrates the deployment of a containerized microservices application on AWS cloud infrastructure. The application is divided into multiple independent services (User Service, Product Service, and Order Service) and deployed using Docker containers on Amazon ECS with Fargate. The services are stored in Amazon ECR, orchestrated using Amazon ECS, and accessed through an Application Load Balancer with path-based routing. A PostgreSQL database on Amazon RDS is used to store data. The objective is to implement a scalable, cloud-native microservices architecture following DevOps practices.

Scope of the Project

The scope of this project includes designing and deploying a microservices architecture using AWS cloud services.

Main objectives include:

- Containerizing applications using Docker
- Storing Docker images in Amazon ECR
- Deploying containers using Amazon ECS Fargate
- Configuring Application Load Balancer for traffic routing
- Implementing health checks using Target Groups
- Connecting application services with Amazon RDS database
- Ensuring high availability and scalability of services

This project demonstrates how modern cloud-native applications are deployed and managed using container orchestration platforms.

Methodologies

The project follows a **DevOps-oriented methodology** using containerization and cloud infrastructure.

Steps followed:

1. Application Development

- Microservices built using Python.

2. Containerization

- Each microservice packaged into a Docker container.

3. Container Registry

- Docker images stored in Amazon Elastic Container Registry (ECR).

4. Container Orchestration

- Services deployed using Amazon ECS with Fargate.

5. Load Balancing

- Application Load Balancer distributes traffic to services.

6. Database Integration

- Amazon RDS PostgreSQL used for persistent storage.

7. Testing & Monitoring

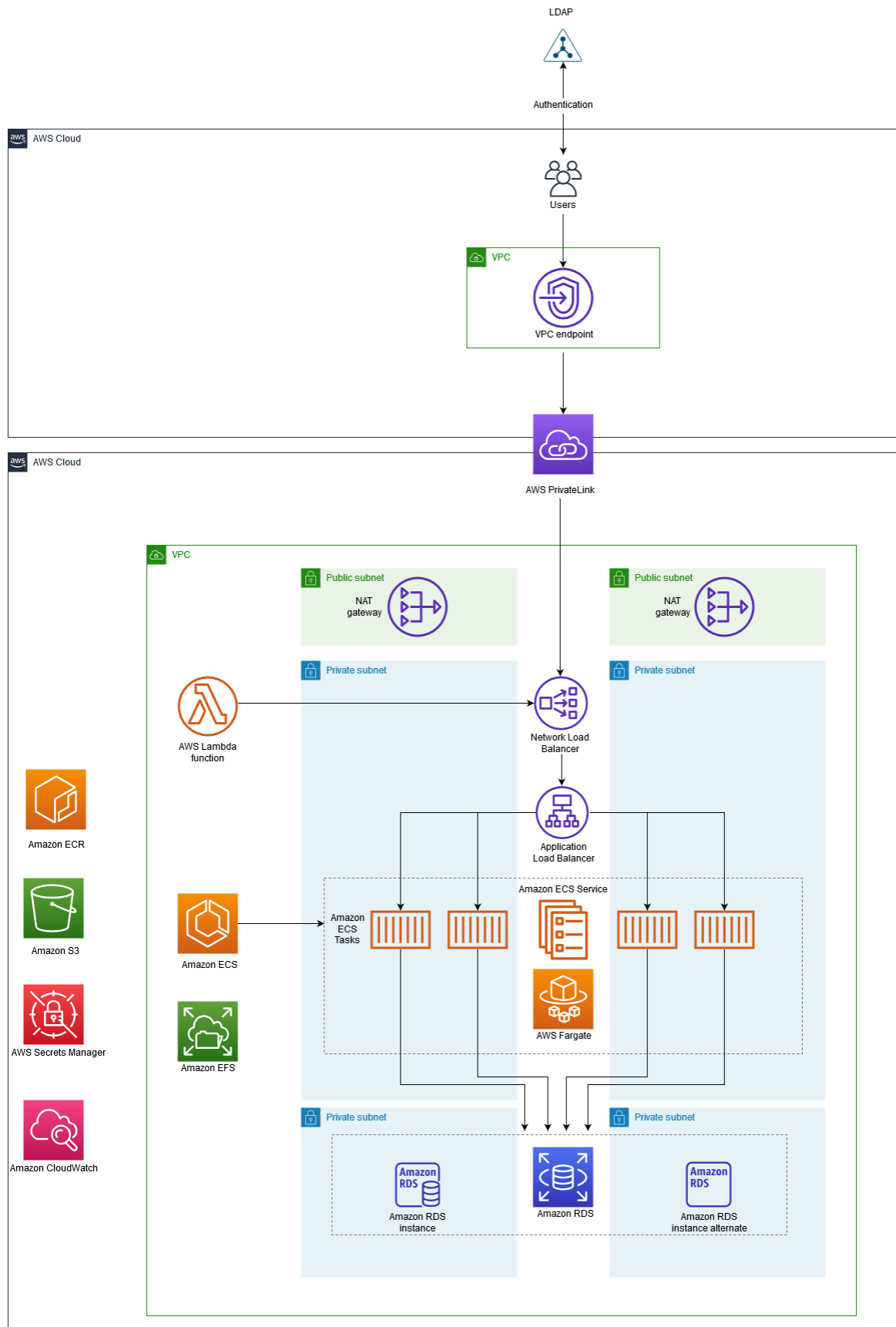
- Health checks used to ensure service availability.

Architecture Diagram / Flow

Architecture flow of the project:

Flow explanation:

1. User sends request to the Application Load Balancer.
2. ALB routes requests based on path rules.
3. ECS services process the requests.
4. Services communicate with RDS database.
5. Response is returned to the user



Explanation of AWS Services Used

Amazon EC2

Used initially to create a development server where Docker was installed and microservices were tested.

Purpose:

- Build Docker images
- Run containers locally
- Test application functionality

Docker

Docker is used to containerize microservices.

Purpose:

- Package application and dependencies
- Ensure consistent runtime environment
- Enable easy deployment to cloud services

Amazon ECR (Elastic Container Registry)

Amazon ECR stores Docker container images.

Purpose:

- Secure storage for Docker images
- Integration with ECS for deployment
- Version management of images

Amazon ECS (Elastic Container Service)

Amazon ECS is used to run containerized applications.

Purpose:

- Container orchestration
- Service management
- Automatic container scheduling

ECS Fargate removes the need to manage servers.

Application Load Balancer (ALB)

ALB distributes incoming traffic to multiple services.

Purpose:

- Load balancing
- Path-based routing
- High availability

Example routing:

/users → User Service

/products → Product Service

/orders → Order Service

Target Groups

Target groups monitor service health and route traffic to containers.

Purpose:

- Health checks
- Traffic distribution
- Service reliability

Amazon RDS (PostgreSQL)

RDS provides a managed relational database service.

Purpose:

- Data storage
- High availability
- Managed backups

Security Groups

Security groups act as virtual firewalls.

Purpose:

- Control inbound and outbound traffic
- Protect cloud infrastructure

Step-by-Step Procedure

Step 1: Launch EC2 Instance

An EC2 instance was launched to prepare the development environment.

Steps:

- Login to AWS Console
- Navigate to EC2 service
- Launch Ubuntu instance
- Configure security group to allow SSH and application ports

Step 2: Install Docker

Docker was installed on the EC2 instance.

Command used:

```
sudo apt update  
sudo apt install docker.io  
docker --version
```

Docker installation was verified using the version command.

Step 3: Build Docker Images

Each microservice was containerized using Docker.

Example command:

```
docker build -t user-service .
```

The container image includes application code and dependencies.

```

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Meghashree (2422-1842-9427) Meghashree

ubuntu@ip-172-31-32-220:~/microservices-project$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~22.04.1
ubuntu@ip-172-31-32-220:~/microservices-project$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
314a2c5a7ebc   d3bb747cfe8f                        "python app.py"         5 hours ago   Up 5 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   musing_rubin
ubuntu@ip-172-31-32-220:~/microservices-project$ docker images
REPOSITORY                                           TAG          IMAGE ID          CREATED          SIZE
242218429427.dkr.ecr.ap-south-1.amazonaws.com/user-service   latest       aa9a998c4a7e     3 hours ago     1.12GB
user-service                                              latest       aa9a998c4a7e     3 hours ago     1.12GB
242218429427.dkr.ecr.ap-south-1.amazonaws.com/order-service   latest       21e5b66e232d     5 hours ago     1.1GB
order-service                                           latest       21e5b66e232d     5 hours ago     1.1GB
product-service                                           latest       c13feefb416      5 hours ago     1.1GB
242218429427.dkr.ecr.ap-south-1.amazonaws.com/product-service latest       c13feefb416      5 hours ago     1.1GB
242218429427.dkr.ecr.ap-south-1.amazonaws.com/user-service   <none>       d3bb747cfe8f     5 hours ago     1.1GB
python                                                  3.9         bcd3da597491     4 months ago    1.09GB
ubuntu@ip-172-31-32-220:~/microservices-project$ tree
.
├── order-service
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
├── product-service
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
└── user-service
    ├── Dockerfile
    ├── app.py
    └── requirements.txt

3 directories, 9 files
ubuntu@ip-172-31-32-220:~/microservices-project$

```

Step 4: Test Docker Containers

Containers were tested locally.

Command:

```
docker run -p 5000:5000 user-service
```

Application was tested using browser or curl.

Step 5: Push Images to Amazon ECR

Docker images were pushed to ECR repositories.

Steps:

- Create repositories in Amazon ECR
- Authenticate Docker with AWS
- Tag images
- Push images to repository

The screenshot shows the Amazon Elastic Container Registry (ECR) console, specifically the 'Private registries' section. The left sidebar shows the navigation menu with 'Private registry' selected. The main content area displays a table of private repositories. At the top, there are buttons for 'View push commands', 'Delete', 'Actions', and a prominent orange 'Create repository' button. A search bar is provided for finding repositories by substring.

Repository name	URI	Created at	Tag immutability	Encryption type
order-service	242218429427.dkr.ecr.ap-south-1.amazonaws.com/order-service	March 06, 2026, 20:14:16 (UTC+05.5)	Mutable	AES-256
product-service	242218429427.dkr.ecr.ap-south-1.amazonaws.com/product-service	March 06, 2026, 20:13:49 (UTC+05.5)	Mutable	AES-256
user-service	242218429427.dkr.ecr.ap-south-1.amazonaws.com/user-service	March 06, 2026, 20:12:06 (UTC+05.5)	Mutable	AES-256

Step 6: Create ECS Cluster

An ECS cluster was created to manage container services.

Steps:

- Navigate to ECS console
- Create cluster
- Select Fargate launch type

The screenshot shows the Amazon Elastic Container Service (ECS) console, specifically the 'Clusters' section. The left sidebar shows the navigation menu with 'Clusters' selected. The main content area displays a table of ECS clusters. At the top, there are buttons for 'Last updated' and a prominent orange 'Create cluster' button. A search bar is provided for finding clusters.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Capacity provider
microservices-cluster	3	1 Pe... 3 Ru...	0 EC2	Default	No default found

Step 7: Create Task Definitions

Task definitions define how containers run.

Information included:

- Docker image location
- CPU and memory allocation
- Container ports

Amazon Elastic Container Service > Task definitions

Task definitions (3) info

Filter task definitions

Filter status: Active

Task definition	Status of last revision
order-service-task	Active
product-service-task	Active
user-service-task	Active

Step 8: Deploy ECS Services

Services were created for each microservice.

Services deployed:

- User Service
- Product Service
- Order Service

Each service runs containers using Fargate.

Amazon Elastic Container Service > Clusters > microservices-cluster > Services

microservices-cluster

Cluster overview

ARN: arn:aws:ecs:ap-south-1:242218429:427:cluster/microservices-cluster

Status: Active

CloudWatch monitoring: Default

Registered container instances: -

Services: Draining: -, Active: 3

Tasks: Pending: 1, Running: 3

Services (3) info

Filter services by value

Filter launch type: Any launch type

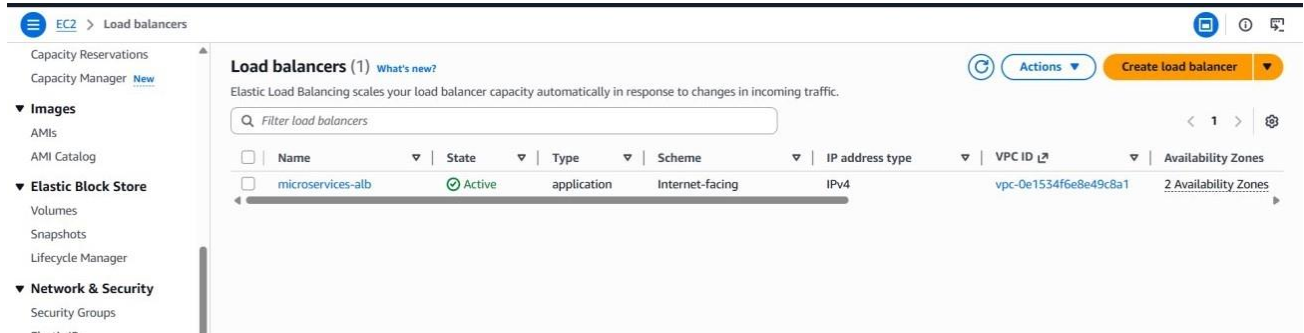
Filter scheduling strategy: Any scheduling strategy

Filter resource management type: Any resource management type

Service name	ARN	Status	Schedu...	Lau...	Task de...	Deployments and tasks
order-service-task-service-w2mgb7vd	arn:aws:ecs:ap-s...	Active	REPLICA	-	order-serv...	1/1 Task
product-service	arn:aws:ecs:ap-s...	Active	REPLICA	-	product-s...	1/1 Task
user-service-task-service-l0643jxd	arn:aws:ecs:ap-s...	Active	REPLICA	-	user-servi...	1/1 Task

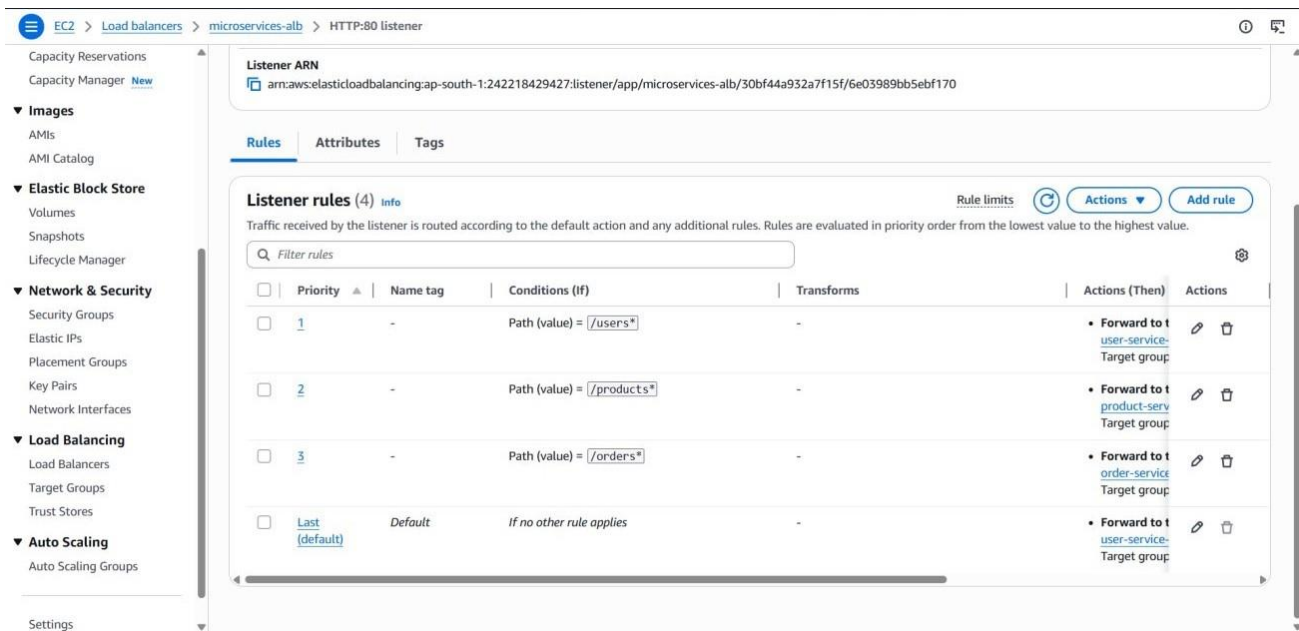
Step 9: Configure Application Load Balancer

An Application Load Balancer was created.



Configuration:

- Internet-facing load balancer
- HTTP listener on port 80



EC2 > Load balancers > microservices-alb

Capacity Reservations
Capacity Manager [New](#)

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Settings

Introducing ALB target optimizer
Target optimizer lets you enforce a maximum number of requests per target using an ALB-provided agent, improving success rates, latency, and efficiency. [Learn more](#)

microservices-alb [Refresh](#) [Actions](#)

▼ Details

Load balancer type Application	Status Active	VPC vpc-0e1534f6e8e49c8a1	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone ZP97RAFLXTNZK	Availability Zones subnet-0ad262f9a5b4f053f ap-south-1b (aps1-az3) subnet-05b963432317ae011 ap-south-1a (aps1-az1)	Date created March 6, 2026, 22:53 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:ap-south-1:242218429427:loadbalancer/app/microservices-alb/30bf44a932a7f15f		DNS name Info microservices-alb-1939454003.ap-south-1.elb.amazonaws.com (A Record)	

[Listeners and rules](#) [Network mapping](#) [Resource map](#) [Security](#) [Monitoring](#) [Integrations](#) [Attributes](#) [Capacity](#) [Tags](#)

Listeners and rules (1) [Info](#) [Refresh](#) [Manage rules](#) [Manage listener](#) [Add listener](#)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

[<](#) [1](#) [>](#) [Settings](#)

Target groups created for each service

ORDER-SERVICE

EC2 > Target groups > order-service-tg

RESET VIEW TO DEFAULTS

Dedicated Hosts
Capacity Reservations
Capacity Manager [New](#)

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

Details

arn:aws:elasticloadbalancing:ap-south-1:242218429427:targetgroup/order-service-tg/a253aa44f9fe4f19

Target type: IP

Protocol : Port: HTTP: 5002

Protocol version: HTTP1

VPC: [vpc-0e1534f6e8e49c8a1](#)

IP address type: IPv4

Load balancer: [microservices-alb](#)

1 Total targets	1 Healthy 0 Anomalous	0 Unhealthy	0 Unused	0 Initial	0 Draining
--------------------	-----------------------------	----------------	-------------	--------------	---------------

► Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) [info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	IP address	Port	Zone	Health status	Health status details	Admini...	Overri...	Anomaly detectio...
☐	172.31.14.98	5002	ap-south-1...	Healthy	-	☐ No override	No overri...	Normal

PRODUCT-SERVICE

EC2 > Target groups > product-service-tg

RESET VIEW TO DEFAULTS

Dedicated Hosts
Capacity Reservations
Capacity Manager [New](#)

▼ Images
AMIs
AMI Catalog

▼ Elastic Block Store
Volumes
Snapshots
Lifecycle Manager

▼ Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

▼ Load Balancing
Load Balancers
Target Groups
Trust Stores

▼ Auto Scaling
Auto Scaling Groups

product-service-tg

arn:aws:elasticloadbalancing:ap-south-1:242218429427:targetgroup/product-service-tg/396d4e75b57211fd

Target type: IP

Protocol : Port: HTTP: 5001

Protocol version: HTTP1

VPC: [vpc-0e1534f6e8e49c8a1](#)

IP address type: IPv4

Load balancer: [microservices-alb](#)

1 Total targets	1 Healthy 0 Anomalous	0 Unhealthy	0 Unused	0 Initial	0 Draining
--------------------	-----------------------------	----------------	-------------	--------------	---------------

► Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) [info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	IP address	Port	Zone	Health status	Health status details	Admini...	Overri...	Anomaly detectio...
☐								

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

USER-SERVICE

EC2 > Target groups > user-service-tg

Details
 arn:aws:elasticloadbalancing:ap-south-1:242218429427:targetgroup/user-service-tg/6ec09631b2eee1e0

Target type IP	Protocol : Port HTTP: 5000	Protocol version HTTP1	VPC vpc-0e1534f6e8e49c8a1
IP address type IPv4	Load balancer microservices-alb		

1 Total targets 1 Healthy 0 Unhealthy 0 Unused 0 Initial 0 Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)
 Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets Monitoring Health checks Attributes Tags

Registered targets (1) [Info](#) Anomaly mitigation: **Not applicable** [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	IP address	Port	Zone	Health status	Health status details	Admini...	Overri...	Anomaly detection...
☐	172.31.38.148	5000	ap-south-1...	Healthy	-	No override.	No overri...	Normal

Step 10: Configure Path-Based Routing

Routing rules were configured:

/users → user-service

Amazon Elastic Container Service > Clusters > [microservices-clusterr](#) > Services > [user-service-task-service-10643jxd](#) > Health

Amazon Elastic Container Service

Express Mode
[Clusters](#)
[Namespaces](#)
[Task definitions](#)
[Account settings](#)

[Amazon ECR](#)
[Repositories](#)

[AWS Batch](#)

[Online learning workshop](#)
[Documentation](#)
[Discover products](#)
[Subscriptions](#)

[Tell us what you think](#)

user-service-task-service-10643jxd [Info](#)

Last updated March 7, 2026, 00:36 (UTC+5:30) [Delete service](#) [Update service](#)

Service overview [Info](#)

Status: [Active](#) Tasks (1 Desired): 0 Pending | 2 Running Task definition: [revision user-service-task:2](#) Deployment status: [In progress](#)

[Health and metrics](#) Tasks Logs Deployments Events Configuration and networking Service auto scaling Event history

Status [Info](#)

Service name: [user-service-task-service-10643jxd](#) Service ARN: [arn:aws:ecs:ap-south-1:242218429427:service/microservices-cluster/user-service-task-service-10643jxd](#) Deployments current state: 2 Failed tasks, 1 Completed task Created at: March 6, 2026, 20:52 (UTC+5:30)

Health check grace period: 0 seconds

Load balancer target health

Container name:port	Target group	Target health	Load balancer	Load balancer type
user-service:5000	user-service-tg Details	1 Healthy 0 Unhealthy	microservices-alb	Application Load Balancer

/products → product-service

Amazon Elastic Container Service > Clusters > microservices-clusterr > Services > product-service > Health

product-service Info Last updated March 7, 2026, 00:36 (UTC+5:30) [Delete service](#) [Update service](#)

Service overview Info

Status Active Tasks (1 Desired) 0 Pending | 1 Running Task definition: revision [product-service-task:1](#) Deployment status Success

[Health and metrics](#) Tasks Logs Deployments Events Configuration and networking Service auto scaling Event history

Status Info

Service name [product-service](#) Service ARN [arn:aws:ecs:ap-south-1:242218429427:service/microservices-cluster/product-service](#) Deployments current state 1 Completed task Created at [March 6, 2026, 21:39 \(UTC+5:30\)](#)

Health check grace period 0 seconds

▼ Load balancer target health

Container name:port	Target group	Target health	Load balancer	Load balancer type
product-service:5001	product-service-tg Details	1 Healthy 0 Unhealthy	microservices-alb	Application Load Balancer

Health

/orders → order-service

Amazon Elastic Container Service > Clusters > microservices-clusterr > Services > order-service-task-service-w2mgb7vd > Health

order-service-task-service-w2mgb7vd Info Last updated March 7, 2026, 00:36 (UTC+5:30) [Delete service](#) [Update service](#)

Service overview Info

Status Active Tasks (1 Desired) 0 Pending | 1 Running Task definition: revision [order-service-task:1](#) Deployment status Success

[Health and metrics](#) Tasks Logs Deployments Events Configuration and networking Service auto scaling Event history

Status Info

Service name [order-service-task-service-w2mgb7vd](#) Service ARN [arn:aws:ecs:ap-south-1:242218429427:service/microservices-cluster/order-service-task-service-w2mgb7vd](#) Deployments current state 1 Completed task Created at [March 6, 2026, 21:44 \(UTC+5:30\)](#)

Health check grace period 0 seconds

▼ Load balancer target health

Container name:port	Target group	Target health	Load balancer	Load balancer type
order-service:5002	order-service-tg Details	1 Healthy 0 Unhealthy	microservices-alb	Application Load Balancer

Health

This allows multiple services to run behind a single load balancer.

Step 11: Create Amazon RDS Database

A PostgreSQL database was created using RDS.

Steps:

- Select PostgreSQL engine
- Configure credentials
- Attach security groups
- Connect services to database

The screenshot shows the AWS Management Console interface for the 'microservices-db' database instance. The left sidebar contains navigation links for Aurora and RDS, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations (0), and Certificate update.

The main content area displays the 'microservices-db' instance details under the 'Connectivity & security' tab. The 'Summary' section shows:

- DB identifier:** microservices-db
- Status:** Available
- Role:** Instance
- Engine:** PostgreSQL
- Region & AZ:** ap-south-1a
- CPU:** 5.08%
- Class:** db.t4g.micro
- Current activity:** 0.00 sessions

The 'Connect using' section offers three options:

- Code snippets:** Use when connecting through SDK, APIs, or third-party tools including agents. (Selected)
- CloudShell:** Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- Endpoints:** Use when connecting through any IDE interface.

The 'Internet access gateway' is set to 'Disabled'. The 'IAM Authentication' is also 'Disabled'.

The 'Programming language' is set to 'PSQL (macOS)' and the 'Endpoint type' is 'Instance endpoint'.

The 'Connection steps' section provides the following commands to be pasted into a terminal:

```
1 export RDSHOST="microservices-db.crw40sgsg30d.ap-south-1.rds.amazonaws.com"
2 psql "host=$RDSHOST port=5432 dbname=postgres user=dbadmin sslmode=verify-full sslrootcert=/certs/global-bundle.pem password=<Enter_DB_Password>"
```

Step 12: Test the Application

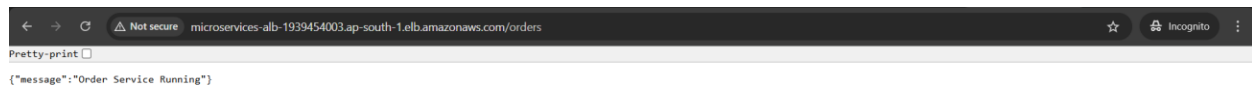
Services were tested using the ALB DNS endpoint

Successful responses confirmed correct deployment.

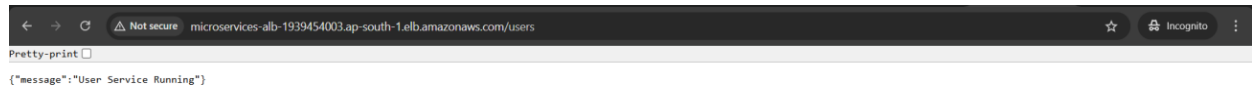
<http://ALB/products>



http://ALB/orders



http://ALB/users



Troubleshooting

Issue 1: 503 Service Temporarily Unavailable

Method used:

Checked target group health.

Solution:

Attached ECS service to target group and verified health check path.

Issue 2: Container Not Registering in Target Group

Method used:

Checked ECS service configuration.

Solution:

Updated service with correct load balancer settings.

Issue 3: Health Check Failure

Method used:

Checked health check path and container port.

Solution:

Updated path to correct API endpoint.

Issue 4: Security Group Blocking Traffic

Method used:

Reviewed security group rules.

Solution:

Allowed inbound HTTP traffic on port 80.

Conclusion

This project successfully demonstrates the deployment of a **microservices-based application using Docker and AWS cloud services**.

The architecture provides scalability, flexibility, and high availability using container orchestration and load balancing. By leveraging services such as **Amazon ECS Fargate, Amazon ECR, Application Load Balancer, and Amazon RDS**, the application can be deployed and managed efficiently without managing underlying servers.

The project highlights the benefits of **containerization and cloud-native architectures**, making it an effective implementation of modern DevOps practices